

BlackHaven / Atlas Masa App Summary

One-page summary based only on repo evidence in `/Users/avrohom/Downloads/BlackHaven`.

What It Is

A multi-platform, local-first AI app ecosystem with web plus native iOS, macOS, Android, and Windows clients. Repo docs describe the website as the entry/sales layer and the native apps as the actual planning, memory, queue, and execution product.

Who It's For

Primary persona: people who want privacy-conscious help organizing life, work, and mobility with local-first AI. A narrower named ICP or demographic profile was **Not found in repo**.

What It Does

- Command Center views for daily, mid-term, and long-horizon planning.
- Adaptive survey/onboarding for profile, intent, and mobility capture.
- Notes, memory ingestion, and local memory wipe controls.
- Prompt queue with local reasoning workers and fallback behavior.
- Execution streams that turn plans into next actions and suggestions.
- Apple, Google, and passkey-based access flows across apps.
- Shared backend plus optional local LLM runtime across platforms.

How It Works

UI layer: Next.js website at repo root (`app/`, `components/`) plus native apps in `ios-app/`, `macos-app/`, `android-app/`, and `windows-app/`.

Service layer: Rust `atlas-concierge/` workspace with crates for core models/policies, retrieval, ML, agents, storage, observability, API, CLI, and tests. The API uses Axum and supports in-memory or SQLite persistence.

Client/runtime flow: user input enters web/native UI, then goes either to local runtime or the shared backend (`/v1/chat`, auth, memory, billing). Android docs show `Room/DataStore/WorkManager`; Windows docs show encrypted local JSON state + DPAPI; Apple docs show runtime config in `UserDefaults` and secrets in `Keychain`.

Local-first inference: repo docs show local LLM endpoints or managed Ollama/Rust reasoners first, with deterministic fallback when services are unavailable.

How To Run

- From repo root: ``npm install`, `cp .env.example .env.local`, `npm run dev``, then open ``http://localhost:3000``.
- Optional backend: ``cd atlas-concierge`, `cp .env.example .env`, load env vars, run `cargo run -p atlas-api`, then check `/health` on port `8080``.
- Native app launch steps are documented separately in the platform READMEs.
- A single full-stack bootstrap command was **Not found in repo**.

Evidence: `README.md`, `package.json`, `atlas-concierge/README.md`, `atlas-concierge/docs/RUNBOOK.md`, and platform READMEs.